

HCMS Recruitment Suite: An AI-Assisted, Workflow-Driven Human Capital Management Platform

LLM-Based CV Screening and Job-Description Generation Fused with Requisition Approvals, Interview Evaluation, Employment Proposals, and Digitally Signed Offers

Bendi Karthikeya

B.Sc. (Honours) Data Science and Artificial Intelligence

Indian Institute of Technology Guwahati (Online)

Student ID: 23035010310 | k.bendi@op.iitg.ac.in

Course: DA 377, Internship-I | Trimester VII

Internship at FIT - FIRST Information Technology, Muscat, Oman (Remote)

Abstract: Hiring in mid-size enterprises is still coordinated through e-mail threads, spreadsheets, and wet-ink signatures, making the recruitment funnel slow, unauditible, and prone to compliance gaps. This work presents the *HCMS Recruitment Suite*, a server-rendered Human Capital Management platform that digitizes the complete hiring lifecycle: manpower requisition with rule-based multi-step approvals, a candidate pipeline with CV intake, structured panel-based interview evaluation across configurable rounds, a non-contractual Employment Proposal stage with role-resolved sequential sign-off chains, and offer-letter generation with embedded electronic signatures through DocuSign and Adobe Acrobat Sign. At its core sits an AI screening engine: candidate documents are parsed and evaluated by a Large Language Model against the job requirement using a structured 8-criterion rubric (80-point scale), with a resilient three-tier inference chain (Groq-served Llama-3.1-8B as primary, DeepSeek via OpenRouter as secondary, and a deterministic regex extractor as offline fallback) producing matching scores, skill-gap analysis, and threshold-based auto-shortlisting; a companion LLM generator drafts structured job descriptions. The system is built as a Django 4.2 modular monolith with a dedicated presentation layer (the `ui` application), comprising 66 routes, 83 view functions, and 42 templates ($\approx 19,400$ lines of presentation code). A sequential-approval engine guarantees that exactly one approver is actionable at any time, producing a tamper-evident audit trail from requisition to signed offer. The result demonstrates that LLM-based screening and a full enterprise hiring workflow, traditionally fragmented across several SaaS tools, can be delivered as a single coherent, auditable web platform.

Index Terms: Large Language Models, AI CV Screening, Prompt Engineering, Structured Output, Human Capital Management, Workflow Automation, Electronic Signatures, Django.

I. INTRODUCTION

A. Internship Context

This work was carried out during my internship as a *Full Stack Generative AI Intern* at **FIT - FIRST Information Technology** (Muscat, Oman; remote), where I build production-grade applications for the company's clients. The

platform presented in this report, the *HCMS Recruitment Suite*, is one such client project, which I designed and implemented end-to-end in Python and Django with an LLM-based CV-screening engine. The broader internship spans a wider technology stack (React, TypeScript, Node.js/Express, Voice AI via VAPI, the WhatsApp Business API, retrieval-augmented generation, OAuth-based CRM integrations, and Drizzle ORM); this report focuses specifically on the HCMS Recruitment Suite and its AI screening component.

B. Problem Statement & Motivation

Recruitment is among the most process-heavy functions in an enterprise: a single hire typically crosses six or more organizational roles (requester, department head, HR, finance, legal, executive management) and produces a dozen artifacts: requisition forms, job descriptions, interview scorecards, proposals, and offer letters. Research on electronic HRM consistently reports that paper-driven hiring inflates cycle time and erodes auditability [1]. Generic HR SaaS products address fragments of this funnel, but stitching them together loses the one property compliance teams care most about: a *single, ordered, attributable trail* of who approved what, when, and with which signature.

The gap this project addresses is therefore not a missing algorithm but a missing *system*: an open, self-hostable platform in which the entire hiring funnel, from "we need a person" to a counter-signed offer letter, lives in one data model, one permission model, and one audit trail.

C. Core Contributions

1. Unified Hiring Data Model: Requisitions, candidates, interviews, proposals, and offers as one relational schema with company-scoped multi-tenant isolation and field-level audit logging.

2. LLM Screening with Graceful Degradation: An AI CV-evaluation engine using a structured 8-criterion rubric and a three-tier inference chain (Llama-3.1-8B [2] via Groq → DeepSeek [3] via OpenRouter → deterministic regex), so screening never hard-fails on provider outages.

3. Sequential Approval Engine: A reusable chain-of-responsibility engine in which only the *first pending* step of a chain is actionable ("active"); later steps remain "waiting" until their turn, eliminating out-of-order sign-offs.

4. Role-Resolved Proposal Chains: Organizational roles (HOD, COO, Legal Advisor, GM HR&A, CFO, CEO) are mapped once by HR and resolved to concrete approvers at routing time, so workflows survive personnel changes.

5. Embedded E-Sign Integration: Provider-agnostic electronic signing via DocuSign eSignature [4] and Adobe Acrobat Sign [5] using embedded (captive) signing ceremonies, with the signed PDF folded back into the approval record.

II. SYSTEM DESIGN & DATA MODEL

A. Architecture

HCMS is a Django 4.2 [6] modular monolith. Domain logic lives in focused apps (`recruitment`, `employee`, `notifications`), while the new presentation layer is isolated in a single `ui` app that owns its routes, views, and templates. This separation lets the interface be rebuilt or replaced wholesale without touching workflow logic. Pages are server-rendered; interactivity (dropdown actions, signature capture, modal previews) uses small vanilla-JavaScript handlers rather than a client-side framework, keeping first-paint fast and the deployment surface minimal. PostgreSQL provides storage with connection pooling; sessions secure the web tier and JWT secures the supplementary REST API.

B. The Hiring Funnel as a State Machine

Every hiring artifact carries an explicit status, and transitions are only performed by engine functions, never by ad-hoc field writes. The funnel is:

```
Manpower Requisition → Multi-step Approval →
  Job Posting / JD → Candidate Pipeline →
    Interview Rounds + Panel Evaluation →
      Employment Proposal (role-chain e-sign) →
        Offer Letter (approval chain +
          DocuSign/Adobe) →
            Acceptance & Onboarding Hand-off
```

FIG. 1. THE NINE-STAGE HIRING FUNNEL IMPLEMENTED BY THE SUITE.

Each transition appends a status-log row capturing actor, timestamp, and remarks, which makes the funnel fully replayable for audit.

C. Approval Chain Model

Approval chains for proposals and offers are stored as ordered step rows (`sequence`, `approver`, `status`, `acted_at`, `signature_image`). The display state of a step is *derived*, not stored: the first step with status `pending` is rendered *active*; all subsequent pending steps render as *waiting*. An approver's inbox therefore contains only steps that are genuinely actionable, which is the invariant that prevents out-of-order or duplicate signing.

III. METHODOLOGY & WORKFLOW ENGINES

The suite is organized as five cooperating engines, all operating on the shared data model:

1) Requisition Approval Routing. A submitted manpower requisition is matched against HR-configured approval rules; per-requester custom chains override the default. Matched rules materialize approval rows; approval or rejection notifies the requester by e-mail and, on final approval, auto-creates a public job posting with a shareable slug.

2) AI Screening Engine (LLM-based). When a CV is uploaded, its text is extracted (PyPDF2 / python-docx) and submitted, together with the job requirement, to an LLM through a single engineered prompt that enforces a two-stage protocol: *extract* structured facts (education, positions, languages, licences, total experience computed by summing position durations), then *score* eight rubric criteria (work experience, qualifications, technical skills, career progression, achievements & impact, role relevance, document quality, and industry knowledge) each on a 1–10 scale (80-point total). Inference runs at low temperature (0.3) with JSON-constrained output for deterministic parsing, through a three-tier fallback chain: Groq-served `llama-3.1-8b-instant` [2] as primary, `deepseek-chat` [3] via OpenRouter as secondary, and a rule-based regex extractor as a no-key offline fallback. Decision thresholds map the percentage score to actions ($\geq 70\%$ auto-shortlist, $\geq 60\%$ standby/interview, otherwise reject), and the result (matching score, matched vs. missing skills, per-criterion commentary) is persisted on a `CandidateScreeningProfile`. Screening executes asynchronously in a background thread with a polling endpoint, so the recruiter's UI never blocks on inference; the dashboard ranks the pipeline by matching score. A companion LLM endpoint generates structured job descriptions from a title and seed attributes, closing the loop between what the JD asks for and what the screener scores against.

3) Interview & Evaluation Engine. Interviews are scheduled with n configurable rounds and a panel of evaluators. Each panelist submits criterion-wise scores; the system aggregates them into a panel summary. A candidate becomes promotable only when *every* round is marked complete (computed as the set difference between

interviews having any round and interviews having an incomplete round), at which point the UI swaps the "Promote" action for "Create Proposal."

4) Proposal Engine. The Employment Proposal is a self-contained snapshot of all hire terms (grade, salary structure, contract type) on one model, deliberately denormalized so that what the chain signs is immutable with respect to later candidate edits. Routing resolves a template-specific role chain (e.g., Project Director → HOD → COO → Legal → GM HR&A → CFO → CEO for permanent hires) through the role-assignment table; unassigned roles are skipped and an empty chain degrades safely to a logged draft.

5) Embedded E-Sign Engine. Signing supports two modes. *Inline*: a canvas-based signature pad stores the drawn signature as a base64 image on the approval step. *Provider-based*: the active step's document is rendered to PDF, an envelope/agreement is created with a single embedded signer, and the approver is redirected into the provider's hosted ceremony; on return, the signed PDF is downloaded, its signature page is attached to the step, and the chain advances. DocuSign and Adobe Acrobat Sign sit behind one provider-agnostic interface, so adding a third provider requires only a new adapter module.

IV. TECHNOLOGY STACK

TABLE I. HCMS RECRUITMENT SUITE TECHNOLOGY STACK

Tier	Technology	Primary Function
AI / LLM	Llama-3.1-8B (Groq), DeepSeek (OpenRouter)	CV scoring, skill matching, JD generation
Parsing	PyPDF2, python-docx, regex	CV text extraction + offline fallback
Backend	Django 4.2 (Python)	ORM, routing, auth, workflow engines
Database	PostgreSQL	Relational store, pooled connections
Presentation	Django templates + vanilla JS	Server-rendered responsive UI
E-Sign	DocuSign, Adobe Acrobat Sign	Embedded signing ceremonies
Signatures	signature_pad (canvas)	Inline drawn-signature capture
E-mail	Brevo SMTP	Invites, approvals, offer notices
Audit	django-auditlog	Field-level change history
Deployment	Docker, Google Cloud Run	CI/CD with migration checks

V. IMPLEMENTATION & EVALUATION

A. Scope of the Delivered System

TABLE II. FUNCTIONAL MODULE COVERAGE

Module	Key Screens
Dashboard	KPIs, funnel, pending-action feed
Requisition	Request form, rules, approval inbox
Candidates	List, profile, pipeline, documents, comms
AI Screening	LLM CV scoring, rubric breakdown, re-screen, live status poll
JD Creator	AI-generated structured job descriptions
Interviews	Scheduler, rounds, panel evaluation, final eval
Proposals	Permanent/contractual forms, role map, e-sign
Offers	Creator, templates, approval chain, final review
E-Sign	DocuSign / Adobe embedded signing + return
System	Login, profile, notifications, integrations

B. Workflow Correctness

The central correctness property, *sequential integrity*, was exercised end-to-end on seeded organizations: with a seven-role permanent-hire chain, the system was verified to (i) expose exactly one active signer at every point, (ii) hide the proposal from later approvers' inboxes until their turn, (iii) skip unassigned roles without breaking sequence, and (iv) finalize only after the last signature, at which point the "Create Offer Letter" hand-off appears with proposal terms pre-filled. Rejection at any step freezes the chain and records reviewer feedback. The same invariant holds for the offer-letter chain, including when a step is signed through an external provider rather than the inline pad.

C. AI Screening Behaviour

The screening engine was exercised across the fallback chain: with a Groq key present, CVs return a full rubric breakdown with per-criterion commentary; with the primary revoked, identical requests transparently route to OpenRouter; with no keys at all, the regex tier still extracts skills and experience so the pipeline keeps functioning, only without semantic scoring. JSON-constrained decoding at temperature 0.3 made responses machine-parseable in practice, and the asynchronous design (daemon-thread inference plus a status-poll endpoint) keeps the recruiter UI responsive: the candidate row simply flips from "screening..." to its score and recommendation when inference lands. Threshold mapping ($\geq 70\%$ shortlist, $\geq 60\%$ standby, $< 60\%$ reject) reproduces the organization's existing paper evaluation form, making the AI's decision directly explainable to HR in their own rubric.

D. Qualitative Outcomes

Because every artifact is born digital, the platform yields (a) a complete who/when/what audit trail per hire, (b) zero re-keying between stages, since interview outcomes flow into proposals and proposals pre-fill offers, (c) an AI-pre-ranked pipeline in which recruiters open the highest-matching candidates first, and (d) a hiring funnel observable in real

time from the reports dashboard rather than reconstructed from inboxes.

VI. CONNECTION TO COURSE LEARNING (DA 3022)

The AI component of this project is a direct application of the course *Large Language Model Application Engineering* (DA 3022). The screening engine in `recruitment/cv_screening_ai.py` is built on the course's coverage of advanced prompt engineering: the anatomy of production prompts, chain-of-thought prompting, structured outputs with constrained decoding, instruction hierarchy and guardrails, and token efficiency. Table III maps each studied concept to where it appears in the implementation.

TABLE III. COURSE CONCEPTS (DA 3022) MAPPED TO THE IMPLEMENTATION

Concept learned (DA 3022)	Where it appears in <code>cv_screening_ai.py</code>
Six-part production-prompt anatomy: role, task, context, examples, constraints, output schema	<code>_build_prompt()</code> sets the role ("senior HR evaluator"), a precise task, the CV + job requirement as context, the 8-criterion rubric, explicit constraints, and a full JSON output schema
Chain-of-thought (step-by-step decomposition)	The prompt's STEP 1 extract → STEP 2 score → STEP 3 total → STEP 4 threshold protocol
Structured outputs, JSON mode, constrained decoding	<code>response_format= {"type": "json_object"}</code> on Groq; defensive regex JSON extraction on the OpenRouter path
Probabilistic generation; low temperature for reliability	<code>temperature=0.3</code> for reproducible scoring of the same CV
Instruction hierarchy and guardrails	System-style directives "Return ONLY valid JSON" and "do not speculate" to suppress hallucination
Positive over negative phrasing	Affirmative "Return only JSON" rather than "don't write prose"
Robustness: retry and validation	Per-tier <code>try/except</code> plus post-inference sanity checks (skill-ratio fallback, duration summation)
Token efficiency and cost control	Single focused prompt with a <code>max_tokens</code> cap

The course's later material on LangChain and LangGraph informs the project's roadmap: the current screening is a single engineered prompt with direct API calls (no orchestration framework), and the natural extension is to express the extract-score-decide pipeline as an explicit LangGraph workflow with tool-calling and retrieval.

VII. LIMITATIONS AND FUTURE WORK

The approval engine currently supports strictly sequential chains; parallel ("any 2 of 3") sign-off is not yet expressible, and provider e-sign assumes a single embedded signer per ceremony. On the AI side, scoring relies on prompt-engineered zero-shot inference: rubric scores are reproducible at low temperature but not formally calibrated against human evaluators, and the regex fallback extracts facts without semantic judgment. Future development will focus on:

- **Semantic retrieval & ranking:** complementing prompt-based scoring with embedding-similarity matching between CV and JD, and a learned ranker calibrated on historical hire outcomes.
- **Agentic orchestration (LangGraph):** expressing the extract-score-decide pipeline as an explicit graph with tool-calling and retrieval, building on the LangChain/LangGraph material from DA 3022.
- **Bias & fairness auditing:** systematic evaluation of LLM scores across demographic proxies before the auto-shortlist threshold is trusted unattended.
- **Parallel & quorum approvals:** generalizing the chain model to DAGs with quorum semantics.

VIII. CONCLUSION

The HCMS Recruitment Suite shows that the full enterprise hiring lifecycle (requisition, approval, AI-screened candidate intake, interviewing, proposal, and digitally signed offers) can be delivered as one coherent, self-hostable web platform. Two engineering ideas carry the system. First, model every decision point as an ordered approval row and derive actionability rather than store it, turning an e-mail-driven process into a single auditable state machine. Second, treat the LLM as a fallible component: constrain it to structured JSON against an explainable human rubric, run it asynchronously, and degrade gracefully through cheaper tiers, which is what makes AI screening dependable enough to sit inside a compliance-grade workflow rather than beside it.

REFERENCES

1. S. Strohmeier, "Research in e-HRM: Review and implications," *Human Resource Management Review*, vol. 17, no. 1, pp. 19–37, 2007.
2. A. Grattafiori et al., "The Llama 3 Herd of Models," *arXiv:2407.21783*, 2024.
3. DeepSeek-AI, "DeepSeek-V3 Technical Report," *arXiv:2412.19437*, 2024.
4. DocuSign Inc., "eSignature REST API Reference," developers.docusign.com, 2024.
5. Adobe Inc., "Acrobat Sign REST API v6," opensource.adobe.com/acrobat-sign, 2024.
6. Django Software Foundation, "Django 4.2 Documentation," docs.djangoproject.com, 2023.